Capstone Project

HPC & Cluster Computing

Capstone Project

Course: Operating Systems 4 (HPC) **Module**: EIOSY4A **Week**: 12

Overview

This is it. The final challenge. There are no new lectures, no new theory. For the past 11 weeks, we have built the components of a commercial-grade Supercomputing Centre. Now, you must assemble them into a cohesive, functioning whole.

You are required to:

1. **Provision** the cluster using Warewulf.
2. **Monitor** the cluster using Prometheus and Grafana.
3. **Program** the cluster by writing and submitting a Parallel Matrix Multiplication script.

---

1. The Integration Challenge

In the real world, "installing software" is easy. Making software **interoperate** is hard.

- NFS needs the network to be up.
- The Network (via Warewulf) needs the nodes to boot.
- Slurm needs the nodes to report "Ready".
- The Application (MPI) needs Slurm to allocate resources.

If *one* link in this chain breaks, the application fails. Your job in this Capstone is to be the **Systems Integrator**.

---

2. Grading Rubric

The Capstone Project is worth **100 Marks**. The breakdown is as follows:

Category	Criterion	Marks
Infrastructure (25%)	All nodes (SMS, NFS, CN01, CN02) are powered on and reachable via SSH.	10
	Warewulf reports all nodes as `READY` via `wwctl node list`.	10
	Network isolation is maintained (subnet `10.10.10.x`).	5
Monitoring (25%)	Prometheus is successfully scraping node metrics.	10
	Grafana Dashboard displays real-time CPU/Ram usage for the compute nodes.	15
HPC Task (50%)	Correct `sbatch` script syntax (Nodes, Tasks, Output logs).	10
	Successful execution of Parallel Matrix Multiplication on >1 node.	20
	Code Logic (Demonstrating `MPI_Scatter`, `MPI_Gather`, or equivalent parallel patterns).	20
TOTAL		**100**

---

3. Project Specifications

Your goal is to demonstrate a fully functional "Mini-Cluster" running a real scientific workload.

3.1 Infrastructure

- **Hardware**: 1 Master Node, 4 Compute Nodes (Virtual Machines).
- **Network**: A private cluster network (e.g., `10.0.0.x`) with functional DNS.

3.2 Management

- **Operating System**: **Rocky Linux 9** (x86_64).
  - *Note: While the specific installation guide references OpenEuler (often used in ARM-based Supercomputers), we use Rocky Linux 9 for this Capstone to ensure stability in the VirtualBox x86 environment. The commands (`dnf`, `wwctl`, `scontrol`) are identical.*
- **Framework**: **OpenHPC v3.x** (EL9 compatible).
- **Provisioning**: **Warewulf 4**.
  - Nodes must boot statelessly (RAM).
  - Nodes must automatically receive the correct overlays (Keys, Hosts).

3.3 Scheduler

- **Workload Manager**: **Slurm**.
  - All nodes must be in the `idle` state (not `down` or `drained`).
  - Must have at least one partition (e.g., `debug`).

3.4 Storage

- **Shared Home**: **NFS** exporting `/home` from Master.
  - Compute nodes must mount `/home` from the SMS.
  - *Note: BeeGFS is removed for this Capstone to simplify the architecture.*

3.5 Application

- **Workload**: **Parallel Matrix Multiplication** (Python/MPI).
- **Execution**: Must run via `sbatch` script on at least 2 nodes (4 cores).
- **Verification**: Log file must show successful execution time.

---

3. Troubleshooting Guide (The Cheat Sheet)

When things go wrong (and they will), use this checklist.

3.1 Scenario: "Nodes won't boot"

- **Check DHCP**: `systemctl status dhcpd`. Is it running?
- **Check TFTP**: Can you ping the Master? firewall?
- **Check Warewulf**: `wwctl node list`. Is the HW address correct?

3.2 Scenario: "Slurm shows nodes as DOWN"

- **Check Time**: `date`. If clocks are out of sync by >5 minutes, Slurm panics.
- **Check Munge**: `systemctl status munge`. Munge keys must match exactly on all nodes.
- **Check Slurmd**: Ssh into the node and check `systemctl status slurmd`.

3.3 Scenario: "Application crashes instantly"

- **Check Storage**: Can you see the file? `ls -l /home/user/case`.
- **Check Libraries**: `module list`. Did you load OpenMPI?

Section Checkpoint

Summary:

- **Goal**: Integrate Weeks 1-11.
- **Specs**: Warewulf + Slurm + NFS + Parallel Python.
- **Method**: Debug layer by layer (Network -> OS -> Middleware -> App).

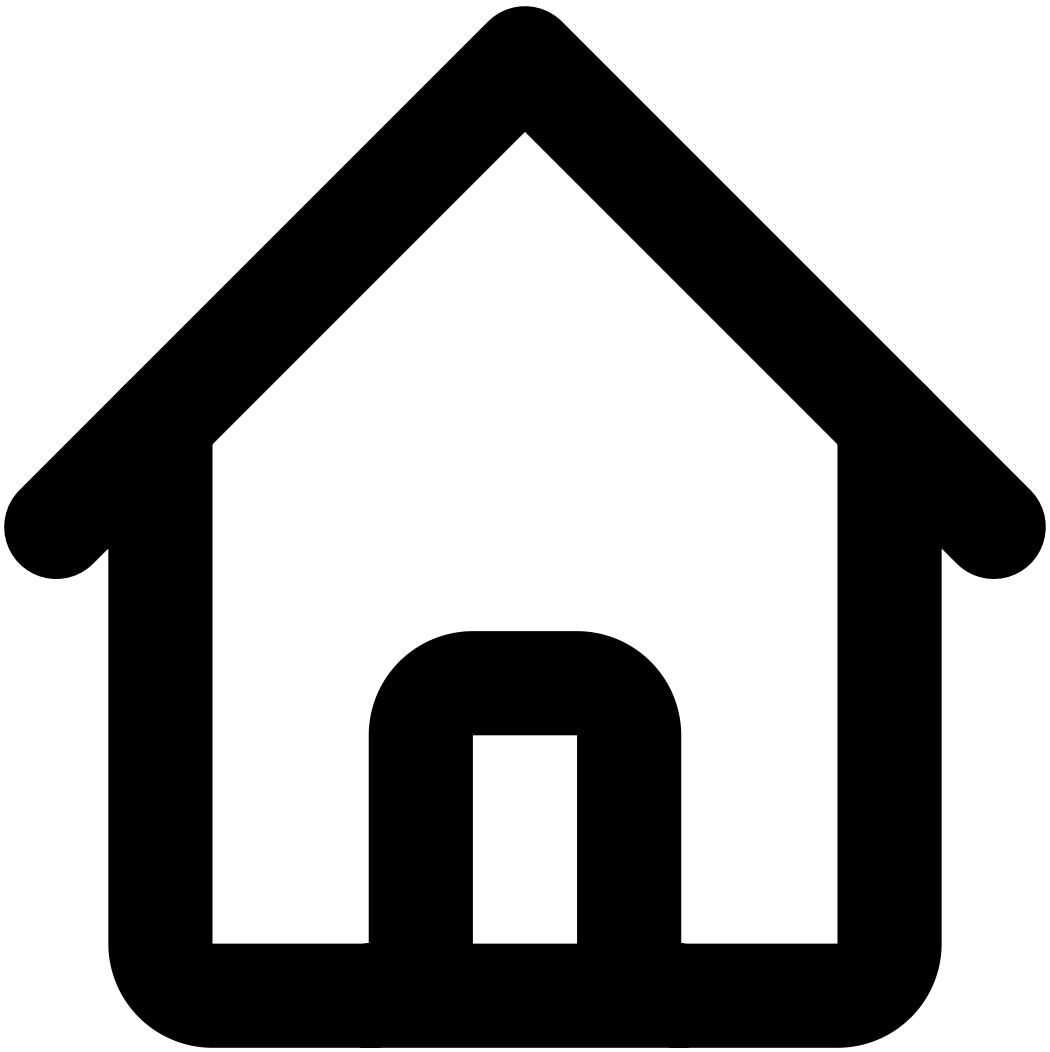
Reflection:

- Why is Time Synchronization (NTP/Chrony) the #1 killer of distributed systems?
- If you could redesign the cluster from Week 1, what would you change?

Resources

- [Your Week 1-11 Notes]
- [Sterling Textbook]

Week 11



Course Index